

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«Ярославский государственный университет им. П.Г. Демидова»

Отчёт по лабораторной работе
«Разработка ПО для вычисления
коэффициентов поглощения $\alpha(S)$ и $\xi(S)$
 n -мерного симплекса»

Избранные задачи вычислительной геометрии

Преподаватель: доктор физико-математических наук,
заведующий кафедрой математического анализа
Невский Михаил Викторович

Выполнили: студенты группы ПМИ-11МО
Слепин Александр Владимирович,
Гаджигасанов Тимур Зейналович.

8 апреля 2026 г., г. Ярославль

Содержание

1	Введение	2
2	Теоретические основы	2
2.1	Базисные многочлены Лагранжа	2
2.2	Осевые диаметры	2
2.3	Коэффициент $\alpha(S)$	3
2.4	Коэффициент $\xi(S)$	3
3	Описание программной реализации	3
3.1	Функции модуля simplex.py	3
3.2	Визуализация	4
4	Исходный код программы	4
4.1	main.py	4
4.2	simplex.py	6
4.3	visualization.py	8
5	Визуализация	12
5.1	Примеры для $\mathbb{R}^1$	12
5.2	Примеры для $\mathbb{R}^2$	15
5.3	Примеры для $\mathbb{R}^3$	21
5.4	Примеры для $\mathbb{R}^5$	27
6	Заключение	27

1 Введение

В учебном пособии М.В. Невского и А.Ю. Ухалова «Избранные задачи анализа и вычислительной геометрии. Часть I» (ЯрГУ, 2020) рассматриваются экстремальные задачи, связанные с числовыми характеристиками конечномерных выпуклых тел, в первую очередь симплексов и кубов. Одними из центральных величин являются коэффициенты поглощения $\alpha(S)$ и $\xi(S)$, определяемые для симплекса $S \subset \mathbb{R}^n$ относительно единичного куба $Q_n = [0, 1]^n$:

$$\alpha(S) = \min\{\sigma > 0 : Q_n \subset \sigma S + t \text{ для некоторого } t \in \mathbb{R}^n\}, \quad \xi(S) = \min\{\sigma \geq 1 : Q_n \subset \sigma S\}.$$

Эти величины имеют важные приложения в вычислительной геометрии и теории интерполяции функций многих переменных.

Целью данной работы являлась разработка программы на языке Python, которая:

- по заданному набору вершин невырожденного n -мерного симплекса S вычисляет $\alpha(S)$ и $\xi(S)$;
- определяет осевые диаметры $d_i(S)$ и концы единственных отрезков максимальной длины, параллельных координатным осям;
- визуализирует симплекс, единичный куб, осевые диаметры и гомотетичные образы αS и ξS для размерностей $n = 1, 2, 3$.

Программа реализована с использованием библиотек NumPy и Matplotlib, строго опирается на формулы и теоремы из указанного пособия.

2 Теоретические основы

2.1 Базисные многочлены Лагранжа

Для невырожденного симплекса S с вершинами $x^{(j)} = (x_1^{(j)}, \dots, x_n^{(j)})$, $j = 1, \dots, n+1$, строится матрица

$$\mathbf{A} = \begin{pmatrix} x_1^{(1)} & \dots & x_n^{(1)} & 1 \\ \vdots & \ddots & \vdots & \vdots \\ x_1^{(n+1)} & \dots & x_n^{(n+1)} & 1 \end{pmatrix},$$

определитель которой $\Delta = \det \mathbf{A}$ связан с объёмом: $\text{vol}(S) = |\Delta|/n!$. Базисные многочлены Лагранжа $\lambda_j(x) = l_{1j}x_1 + \dots + l_{nj}x_n + l_{n+1,j}$ определяются как элементы обратной матрицы $\mathbf{A}^{-1} = (l_{ij})$; они обладают свойствами

$$\lambda_j(x^{(k)}) = \delta_{jk}, \quad \sum_{j=1}^{n+1} \lambda_j(x) = 1, \quad \sum_{j=1}^{n+1} \lambda_j(x)x^{(j)} = x.$$

Эти соотношения подробно описаны в §1.2 пособия (стр. 7–8).

2.2 Осевые диаметры

Для каждого направления координатной оси x_i в симплексе существует единственный отрезок максимальной длины $d_i(S)$, параллельный этой оси. Согласно теореме 1.1 (стр. 10) его длина выражается через элементы обратной матрицы:

$$\frac{1}{d_i(S)} = \frac{1}{2} \sum_{j=1}^{n+1} |l_{ij}|, \quad i = 1, \dots, n.$$

Концы этого отрезка вычисляются по формулам (1.13)–(1.14) (стр. 12):

$$y_-^{(i)} = \sum_{j=1}^{n+1} s_{ij} x^{(j)}, \quad y_+^{(i)} = \sum_{j=1}^{n+1} t_{ij} x^{(j)}, \quad s_{ij} = \frac{|l_{ij}| - l_{ij}}{\sum_k |l_{ik}|}, \quad t_{ij} = \frac{|l_{ij}| + l_{ij}}{\sum_k |l_{ik}|}.$$

2.3 Коэффициент $\alpha(S)$

Из теоремы 1.3 (стр. 20) для $C = Q_n$ получаем

$$\alpha(S) = \sum_{i=1}^n \frac{1}{d_i(S)} = \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^{n+1} |l_{ij}|. \quad (1.36)$$

Эта формула удобна для вычислений, так как требует лишь элементов $\mathbf{A}^{-1}$.

2.4 Коэффициент $\xi(S)$

По определению $\xi(S) = \min\{\sigma \geq 1 : Q_n \subset \sigma S\}$. Из теоремы 1.2 (стр. 16) и леммы 1.4 (стр. 20) следует формула

$$\xi(S) = (n+1) \max_{1 \leq j \leq n+1} \max_{x \in \text{ver}(Q_n)} (-\lambda_j(x)) + 1. \quad (1.25)$$

Для каждого j максимум по вершинам куба достигается в одной из вершин и может быть найден аналитически:

$$\max_{x \in \text{ver}(Q_n)} (-\lambda_j(x)) = \sum_{i: l_{ij} < 0} |l_{ij}| - l_{n+1,j}.$$

Если полученное значение $\xi(S)$ оказывается меньше 1 (что означало бы $Q_n \subset S$), следует вернуть 1.

3 Описание программной реализации

Программа состоит из нескольких модулей:

simplex.py – основные функции для чтения данных, построения матрицы $\mathbf{A}$, вычисления обратной матрицы, осевых диаметров, $\alpha(S)$ и $\xi(S)$.

visualization.py – функции визуализации для размерностей $n = 1, 2, 3$ (отрезок, треугольник, тетраэдр) с отображением единичного куба, симплекса, осевых диаметров и гомотетичных образов αS и ξS .

main.py – точка входа: обрабатывает аргументы командной строки (файл с симплексом), выполняет вычисления и вызывает визуализацию, если размерность поддерживается.

3.1 Функции модуля simplex.py

- **read_simplex(filename)** – читает файл, содержащий в первой строке размерность n , затем $n+1$ строк с координатами вершин. Возвращает n и массив вершин.
- **build_A(vertices)** – строит матрицу $\mathbf{A}$ размера $(n+1) \times (n+1)$ путём добавления столбца единиц к координатам вершин.

- `compute_inverse(A)` – вычисляет A^{-1} с помощью `numpy.linalg.inv`.
- `compute_alpha(A_inv, n)` – вычисляет $\alpha(S)$ по формуле (1.36) как полусумму модулей элементов первых n строк A^{-1} .
- `compute_xi(A_inv, n)` – вычисляет $\xi(S)$ согласно (1.25) с аналитическим определением максимума для каждого j ; возвращает максимум из полученного значения и 1.
- `compute_axial_ends(A_inv, vertices)` – возвращает список пар концов осевых диаметров для каждой координатной оси, используя (1.13)–(1.14).
- `compute_center(vertices)` – центр тяжести симплекса (среднее арифметическое вершин).
- `compute_alpha_center(A_inv, vertices, alpha)` – по формуле (1.44) (стр. 26) вычисляет центр гомотетии x_0 , такой что $Q_n \subset \alpha S + x_0$. Используется для визуализации αS .

3.2 Визуализация

Модуль `visualization.py` содержит функции:

- `plot_1d(vertices, axial_ends, alpha, xi, alpha_center, C_vertices=None)` – для $n = 1$ рисует отрезок $[0, 1]$, симплекс S , осевой диаметр (совпадает с S), ξS (зелёный пункт) и αS (оранжевый пункт) с центром в `alpha_center`.
- `plot_2d(vertices, axial_ends, alpha, xi, alpha_center, C_vertices=None)` – для $n = 2$ рисует квадрат $[0, 1]^2$, треугольник S , осевые диаметры, ξS и при аналогичных условиях αS .
- `plot_3d(vertices, axial_ends, alpha, xi, alpha_center)` – для $n = 3$ рисует каркас куба $[0, 1]^3$, тетраэдр S (полупрозрачные грани), осевые диаметры, контуры ξS (зелёный) и αS (оранжевый пункт).

Все функции используют библиотеку Matplotlib и корректно отображают легенду, заголовков с вычисленными значениями α и ξ .

4 Исходный код программы

4.1 main.py

Листинг 1: main.py

```

1 import sys
2 import time
3
4 from colorama import init, Fore, Style
5 from simplex import read_simplex, build_A, compute_inverse,
6     compute_alpha, compute_xi, compute_axial_ends, \
7     compute_center, compute_alpha_center
8 from visualization import plot_1d, plot_2d, plot_3d
9

```

```

10 def printCol(text, color):
11     print(color + text + Style.RESET_ALL)
12
13
14 def error(text):
15     printCol(text, Fore.RED)
16
17
18 def delimiter():
19     printCol("*" * 100, Fore.YELLOW)
20
21
22 def main():
23     if len(sys.argv) != 2:
24         error("Использование: python main.py <файл_с_координатами>")
25         sys.exit(1)
26
27     init()
28
29     start = time.perf_counter()
30
31     filename = sys.argv[1]
32     try:
33         n, vertices = read_simplex(filename)
34     except Exception as e:
35         error(f"Ошибка чтения файла: {e}")
36         sys.exit(1)
37
38     delimiter()
39     printCol(f"Размерность n = {n}", Fore.BLUE)
40     printCol("Вершины:", Fore.CYAN)
41     for i, v in enumerate(vertices):
42         printCol(f"  x{i + 1} = ({', '.join(str(int(x) if x.is_integer
43             () else x) for x in v)})", Fore.LIGHTCYAN_EX)
44
45     delimiter()
46
47     printCol("Строим матрицу A", Fore.BLUE)
48     A = build_A(vertices)
49     printCol("A:", Fore.CYAN)
50     for row in A:
51         row_str = "  " + " ".join(f"{x:10.3f}" for x in row)
52         printCol(row_str, Fore.LIGHTCYAN_EX)
53
54     delimiter()
55
56     printCol("Ищем обратную матрицу для A", Fore.BLUE)
57     A_inv = compute_inverse(A)
58     printCol("  A      :", Fore.CYAN)
59     for row in A_inv:
60         row_str = "  " + " ".join(f"{x:10.3f}" for x in row)
61         printCol(row_str, Fore.LIGHTCYAN_EX)
62
63     delimiter()
64
65     printCol("Вычисляем (S) и (S)", Fore.BLUE)

```

```

62     alpha = compute_alpha(A_inv, n)
63     xi = compute_xi(A_inv, n)
64     printCol(f"    (S) = {alpha}", Fore.GREEN)
65     printCol(f"    (S) = {xi}", Fore.GREEN)
66     delimiter()
67
68     printCol("Находим осевые диаметры и центр тяжести", Fore.BLUE)
69     axial_ends = compute_axial_ends(A_inv, vertices)
70     center = compute_center(vertices)
71     alpha_center = compute_alpha_center(A_inv, vertices, alpha)
72     printCol(f"Центр тяжести: {center}", Fore.CYAN)
73     delimiter()
74
75     end = time.perf_counter()
76     printCol(f"        Программа отработала за {end - start:.6f} с", Fore
77             .GREEN)
78     delimiter()
79
80     if n > 3:
81         printCol("        Не могу визуализировать для размерности n > 3.",
82                 Fore.GREEN)
83         sys.exit(0)
84
85     printCol("        Визуализация доступна в новом окне.", Fore.GREEN)
86
87     if n == 1:
88         plot_1d(vertices, axial_ends, alpha, xi, alpha_center)
89     elif n == 2:
90         plot_2d(vertices, axial_ends, alpha, xi, alpha_center)
91     elif n == 3:
92         plot_3d(vertices, axial_ends, alpha, xi, alpha_center)
93
94 if __name__ == "__main__":
95     main()

```

4.2 simplex.py

Листинг 2: simplex.py

```

1  import numpy as np
2
3
4  def read_simplex(filename):
5      """
6      Читает файл с координатами вершин симплекса.
7      Формат: первая строка содержит целое число n (размерность),
8      затем n+1 строк, каждая из которых содержит n чисел (координаты вер
9     шины).
10     Возвращает n и массив вершин (n+1, n).
11     """
12     with open(filename, 'r') as f:
13         n = int(f.readline().strip())
14         vertices = []

```

```

14         for _ in range(n + 1):
15             line = f.readline()
16             if not line:
17                 raise ValueError("Недостаточно строк с вершинами")
18             coords = list(map(float, line.split()))
19             if len(coords) != n:
20                 raise ValueError(f"Ожидалось {n} координат, получено {
21                     len(coords)}")
22             vertices.append(coords)
23         return n, np.array(vertices, dtype=float)
24
25 def build_A(vertices):
26     """
27     Строит матрицу A размера (n+1) x (n+1):
28     первые n столбцов - координаты вершин, последний столбец - единицы.
29     """
30     A = np.hstack([vertices, np.ones((vertices.shape[0], 1))])
31     return A
32
33 def compute_inverse(A):
34     """Возвращает обратную матрицу A-1."""
35     return np.linalg.inv(A)
36
37
38 def compute_alpha(A_inv, n):
39     """
40     Вычисляет (S) = 0.5 * sum_{i=1..n} sum_{j=1..n+1} |l_{ij}|
41     где l_{ij} - элементы A_inv (строки i, столбцы j).
42     """
43     # Берём только первые n строк (соответствуют координатам)
44     rows = A_inv[:n, :]
45     return 0.5 * np.sum(np.abs(rows))
46
47
48 def compute_xi(A_inv, n):
49     """
50     Вычисляет (S) = (n+1) * max_j M_j + 1,
51     где M_j = sum_{i: l_{ij}<0} |l_{ij}| - l_{n+1,j}.
52     """
53     M = []
54     for j in range(n + 1):
55         col = A_inv[:n, j] # коэффициенты при переменных
56         free = A_inv[n, j] # свободный член
57         neg_sum = -np.sum(col[col < 0]) # сумма модулей отрицательных
58         M_j = neg_sum - free
59         M.append(M_j)
60     return max(1.0, (n + 1) * max(M) + 1)
61
62
63 def compute_axial_ends(A_inv, vertices):
64     """
65

```

```

66     Для каждой координатной оси  $i$  ( $1..n$ ) возвращает пару точек (концы о
        трезка)
67     максимальной длины, параллельного этой оси.
68     Использует формулы (1.13)-(1.14) из пособия.
69     """
70     n = vertices.shape[1]
71     ends = []
72     for i in range(n):
73         row = A_inv[i, :] #  $l_{i1}$  ...  $l_{i,n+1}$ 
74         s = np.abs(row) - row
75         t = np.abs(row) + row
76         sum_abs = np.sum(np.abs(row))
77         if sum_abs == 0:
78             raise ValueError(f"Нулевая строка {i + 1} в обратной матриц
                e")
79         alpha = s / sum_abs
80         beta = t / sum_abs
81         y_minus = np.sum(alpha[:, np.newaxis] * vertices, axis=0)
82         y_plus = np.sum(beta[:, np.newaxis] * vertices, axis=0)
83         ends.append((y_minus, y_plus))
84     return ends
85
86
87 def compute_center(vertices):
88     """Центр тяжести симплекса (среднее арифметическое вершин)."""
89     return np.mean(vertices, axis=0)
90
91
92 def compute_alpha_center(A_inv, vertices, alpha):
93     """
94     Вычисляет центр гомотетии  $x_0$  для  $(S)$  по формуле (1.44).
95     A_inv - обратная матрица  $A^{-1}$  размера  $(n+1) \times (n+1)$ ,
96     vertices - массив вершин  $(n+1, n)$ ,
97     alpha - значение  $(S)$ .
98     Возвращает массив длины  $n$ .
99     """
100     n = vertices.shape[1]
101     # суммы модулей элементов в столбцах (по первым  $n$  строкам)
102     s_j = np.sum(np.abs(A_inv[:n, :]), axis=0) # вектор длины  $n+1$ 
103     sum_s = np.sum(s_j) #  $= 2 * \alpha$ 
104     denominator = sum_s - 2 #  $= 2 * \alpha - 2$ 
105     if abs(denominator) < 1e-12:
106         raise ValueError("Знаменатель близок к нулю:  $(S)$  1?")
107     numerator = np.zeros(n)
108     for k in range(n):
109         #  $s_j * x_k^{(j)} - 1$ 
110         numerator[k] = np.sum(s_j * vertices[:, k]) - 1
111     x0 = numerator / denominator
112     return x0

```

4.3 visualization.py

```

1 import matplotlib.pyplot as plt
2 from mpl_toolkits.mplot3d.art3d import Poly3DCollection
3 import numpy as np
4
5
6 def plot_1d(vertices, axial_ends, alpha, xi, alpha_center):
7     """1D график с S, осевым диаметром, (S)S и (S)S."""
8     fig, ax = plt.subplots(figsize=(6, 2))
9     ax.grid(True)
10
11     # Границы куба Q1 = [0,1]
12     ax.axvline(0, color='k', linestyle='--', alpha=0.5)
13     ax.axvline(1, color='k', linestyle='--', alpha=0.5)
14
15     # Отрезок S
16     x1, x2 = vertices.flatten()
17     ax.plot([x1, x2], [0, 0], 'b-', linewidth=4, label='S')
18     ax.plot(x1, 0, 'bo')
19     ax.plot(x2, 0, 'bo')
20
21     # Осевой диаметр (совпадает с S, чуть выше)
22     y_minus, y_plus = axial_ends[0]
23     ax.plot([y_minus[0], y_plus[0]], [0.1, 0.1], 'r-', linewidth=2,
24             label='Осевой диаметр')
25
26     # Гомотетичный образ (S)S
27     center = np.mean(vertices, axis=0)
28     scaled_alpha = alpha_center + alpha * (vertices - alpha_center)
29     xa1, xa2 = scaled_alpha.flatten()
30     ax.plot([xa1, xa2], [-0.2, -0.2], 'orange', linestyle=':',
31             linewidth=2, label='(S)S')
32     ax.plot(xa1, -0.2, 'o', color='orange')
33     ax.plot(xa2, -0.2, 'o', color='orange')
34
35     # Гомотетичный образ (S)S
36     scaled_xi = center + xi * (vertices - center)
37     xs1, xs2 = scaled_xi.flatten()
38     ax.plot([xs1, xs2], [-0.1, -0.1], 'g--', linewidth=2, label='(S)S')
39     ax.plot(xs1, -0.1, 'go')
40     ax.plot(xs2, -0.1, 'go')
41
42     ax.set_yticks([])
43     ax.set_xlabel('x')
44     ax.set_title(f'(S) = {alpha:.6f}, (S) = {xi:.6f}', fontsize=20)
45     ax.legend()
46     plt.show()
47
48 def plot_2d(vertices, axial_ends, alpha, xi, alpha_center):
49     """2D график с Q, S, осевыми диаметрами, (S)S и (S)S."""
50     fig, ax = plt.subplots()

```

```

50 ax.grid(True)
51
52 # Квадрат
53 square = np.array([[0, 0], [1, 0], [1, 1], [0, 1], [0, 0]])
54 ax.plot(square[:, 0], square[:, 1], 'k-', lw=1, label='Q ')
55
56 # Симплекс (треугольник)
57 tri = np.vstack([vertices, vertices[0]])
58 ax.plot(tri[:, 0], tri[:, 1], 'b-', lw=2, label='S')
59
60 # Осевые диаметры
61 for i, (ym, yp) in enumerate(axial_ends):
62     ax.plot([ym[0], yp[0]], [ym[1], yp[1]], 'r-', lw=2, label='Осевые диаметры' if i == 0 else '')
63
64 # Гомотетичный образ (S)S
65 center = np.mean(vertices, axis=0)
66 scaled_alpha = alpha_center + alpha * (vertices - alpha_center)
67 scaled_alpha = np.vstack([scaled_alpha, scaled_alpha[0]])
68 ax.plot(scaled_alpha[:, 0], scaled_alpha[:, 1], 'orange', linestyle=':', lw=2, label=f'(S)S')
69
70 # Гомотетичный образ (S)S
71 scaled_xi = center + xi * (vertices - center)
72 scaled_xi = np.vstack([scaled_xi, scaled_xi[0]])
73 ax.plot(scaled_xi[:, 0], scaled_xi[:, 1], 'g--', lw=2, label=f'(S)S')
74
75 ax.set_aspect('equal')
76 ax.set_title(f'(S) = {alpha:.6f}, (S) = {xi:.6f}', fontsize=20)
77 ax.legend()
78 plt.show()
79
80
81 def plot_3d(vertices, axial_ends, alpha, xi, alpha_center):
82     """3D график с кубом Q, тетраэдром, осевыми диаметрами, (S)S и (S)S."""
83     fig = plt.figure()
84     ax = fig.add_subplot(111, projection='3d')
85
86     # Вершины куба [0,1]^3
87     cube_vertices = np.array([[0, 0, 0], [1, 0, 0], [1, 1, 0], [0, 1, 0],
88                                [0, 0, 1], [1, 0, 1], [1, 1, 1], [0, 1, 1]])
89
90     # Рёбра куба
91     edges = [(0, 1), (1, 2), (2, 3), (3, 0), # нижняя грань
92              (4, 5), (5, 6), (6, 7), (7, 4), # верхняя грань
93              (0, 4), (1, 5), (2, 6), (3, 7)] # вертикальные
94
95     for e in edges:
96         ax.plot3D(*zip(cube_vertices[e[0]], cube_vertices[e[1]]), color='k', linewidth=0.5,

```

```

96         label=' Q ' if e == (0, 1) else '')
97
98     # Тетраэдр (грани)
99     faces = [
100         [vertices[0], vertices[1], vertices[2]],
101         [vertices[0], vertices[1], vertices[3]],
102         [vertices[0], vertices[2], vertices[3]],
103         [vertices[1], vertices[2], vertices[3]]
104     ]
105     ax.add_collection3d(Poly3DCollection(faces, alpha=0.2, facecolor='
        cyan', linewidths=1, edgecolor='b', label='S'))
106
107     # Осевые диаметры
108     for i, (ym, yp) in enumerate(axial_ends):
109         ax.plot3D(*zip(ym, yp), color='r', linewidth=2, label='Осевые д
            iamетры' if i == 0 else '')
110
111     # Гомотетичный образ (S)S (контур)
112     center = np.mean(vertices, axis=0)
113     scaled_alpha = alpha_center + alpha * (vertices - alpha_center)
114     alpha_edges = [(0, 1), (0, 2), (0, 3), (1, 2), (1, 3), (2, 3)]
115     tmp = 0
116     for e in alpha_edges:
117         ax.plot3D(*zip(scaled_alpha[e[0]], scaled_alpha[e[1]]), color='
            orange', linestyle=':', linewidth=2,
118                     label=' (S)S' if tmp == 0 else '')
119         tmp += 1
120
121     # Гомотетичный образ (S)S (контур)
122     scaled_xi = center + xi * (vertices - center)
123     tmp = 0
124     for e in alpha_edges:
125         ax.plot3D(*zip(scaled_xi[e[0]], scaled_xi[e[1]]), color='g',
126                     linestyle='--', linewidth=2,
127                     label=' (S)S' if tmp == 0 else '')
128         tmp += 1
129
130     ax.set_xlabel('X')
131     ax.set_ylabel('Y')
132     ax.set_zlabel('Z')
133     ax.set_title(f' (S) = {alpha:.6f}, (S) = {xi:.6f}', fontsize=20)
134     ax.legend()
135     plt.show()

```

5 Визуализация

5.1 Примеры для $\mathbb{R}^1$

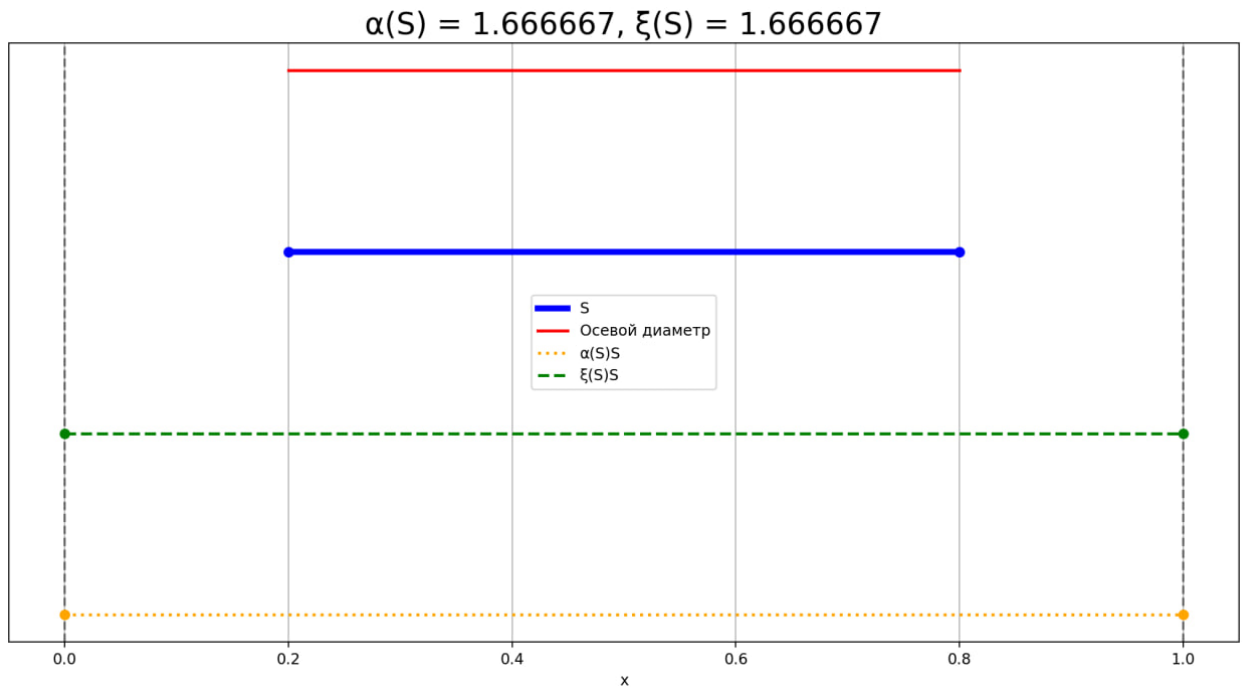


Рис. 1: $S ([0.2, 0.8])$ внутри $Q ([0, 1])$

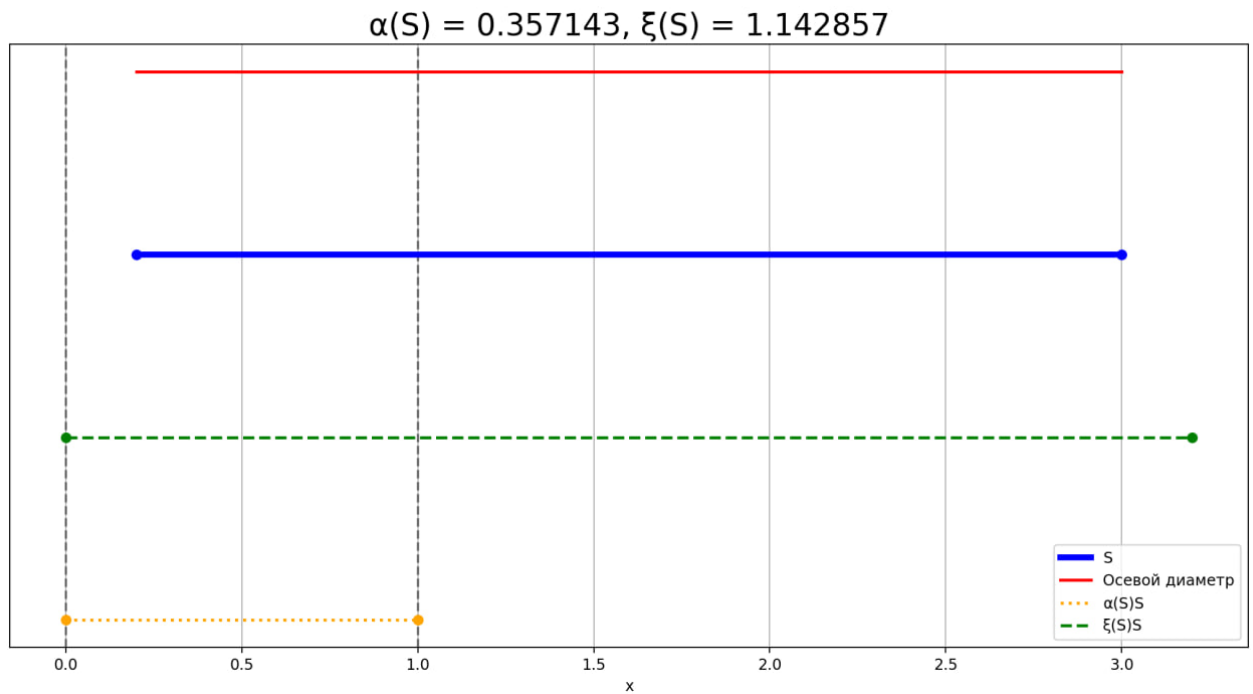


Рис. 2: $S ([0.2, 3])$ частично в $Q_1 ([0, 1])$

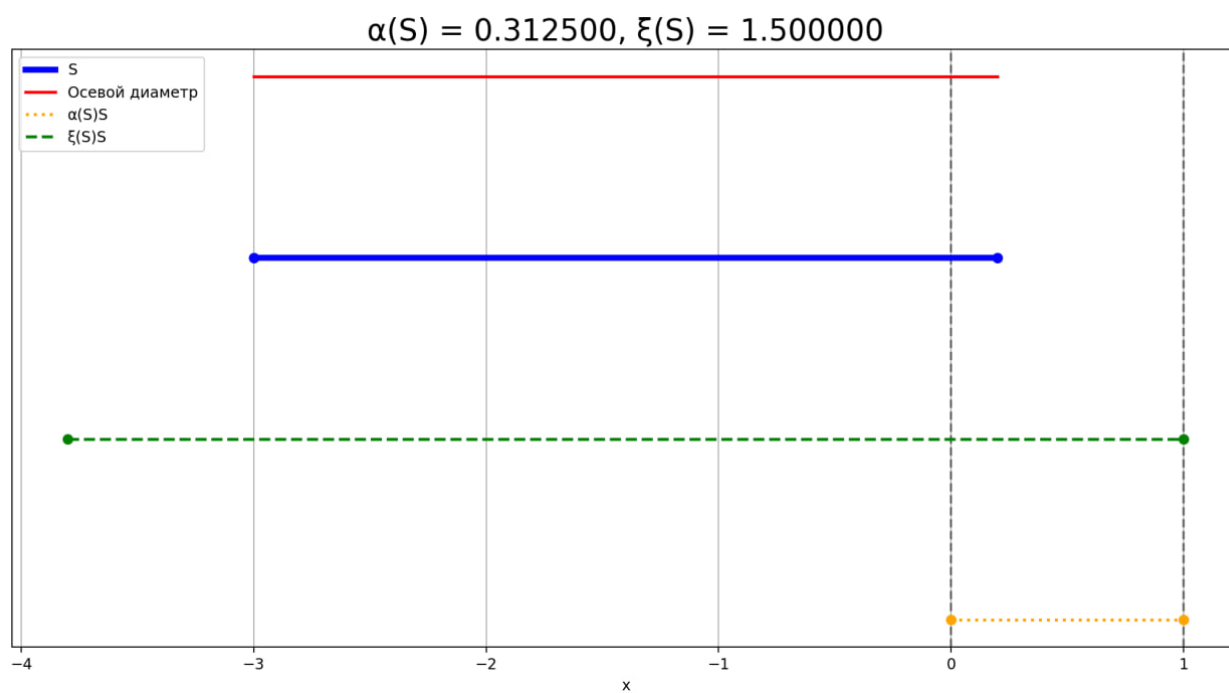


Рис. 3: $S \left([-3, 0.2] \right)$ частично в $Q_1 \left([0, 1] \right)$

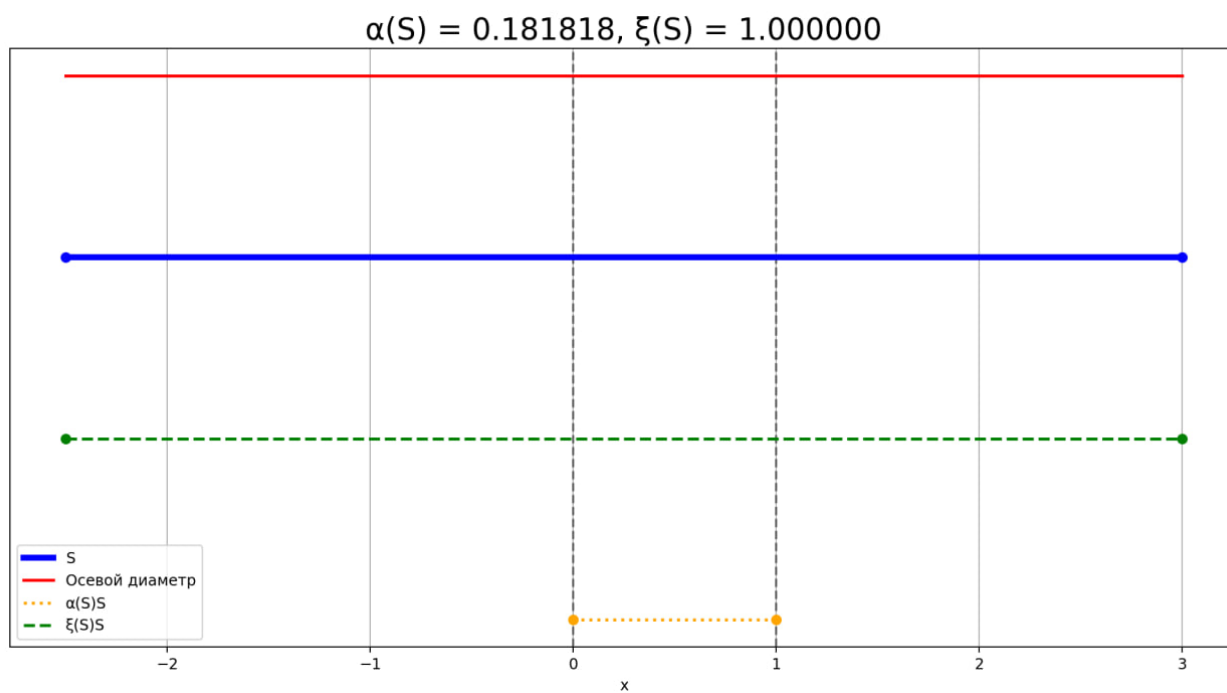


Рис. 4: $S \left([-2.5, 3] \right)$ покрывает $Q_1 \left([0, 1] \right)$

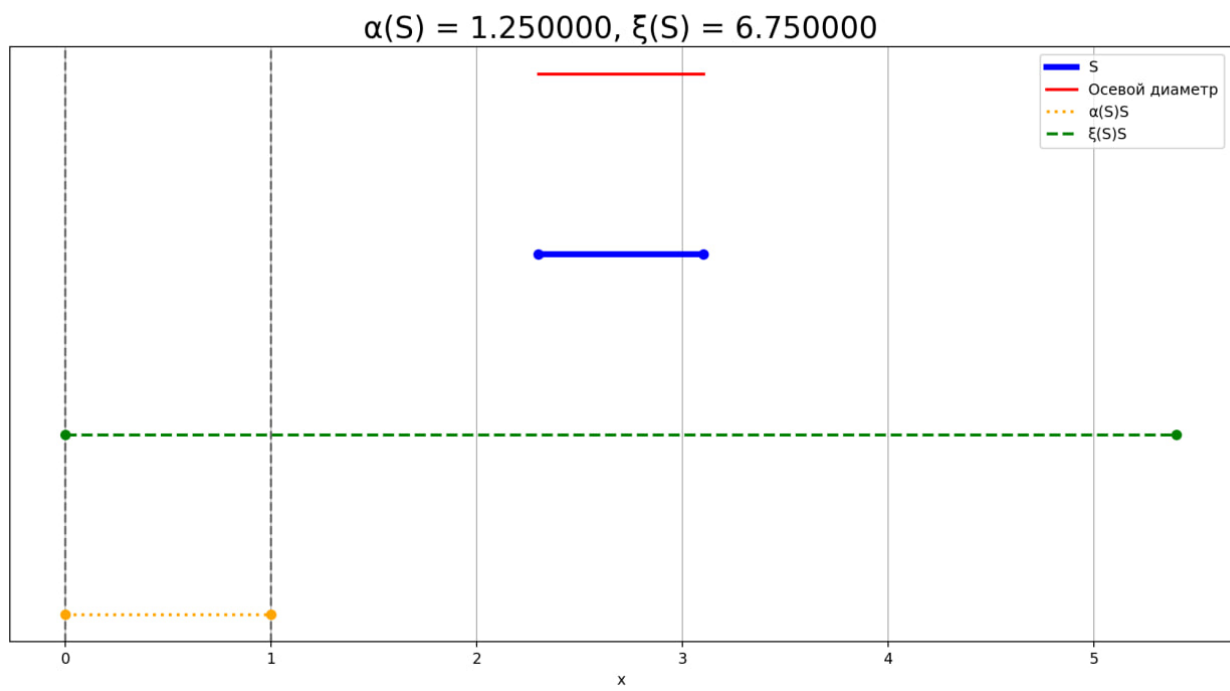


Рис. 5: S $([2.3, 3.1])$ вне Q_1 $([0, 1])$

5.2 Примеры для $\mathbb{R}^2$

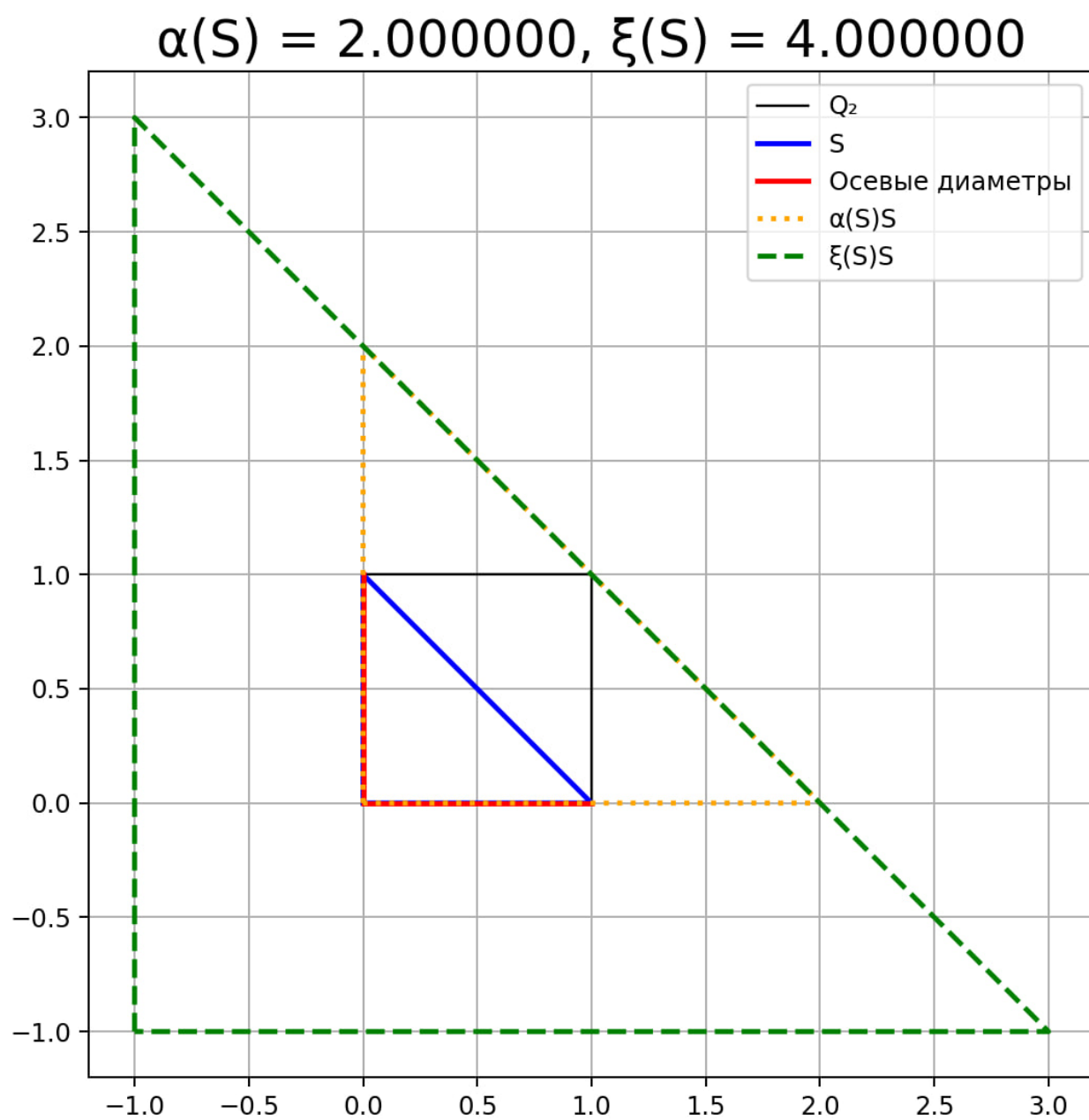


Рис. 6: S внутри Q_2 ($[0, 1]^2$)

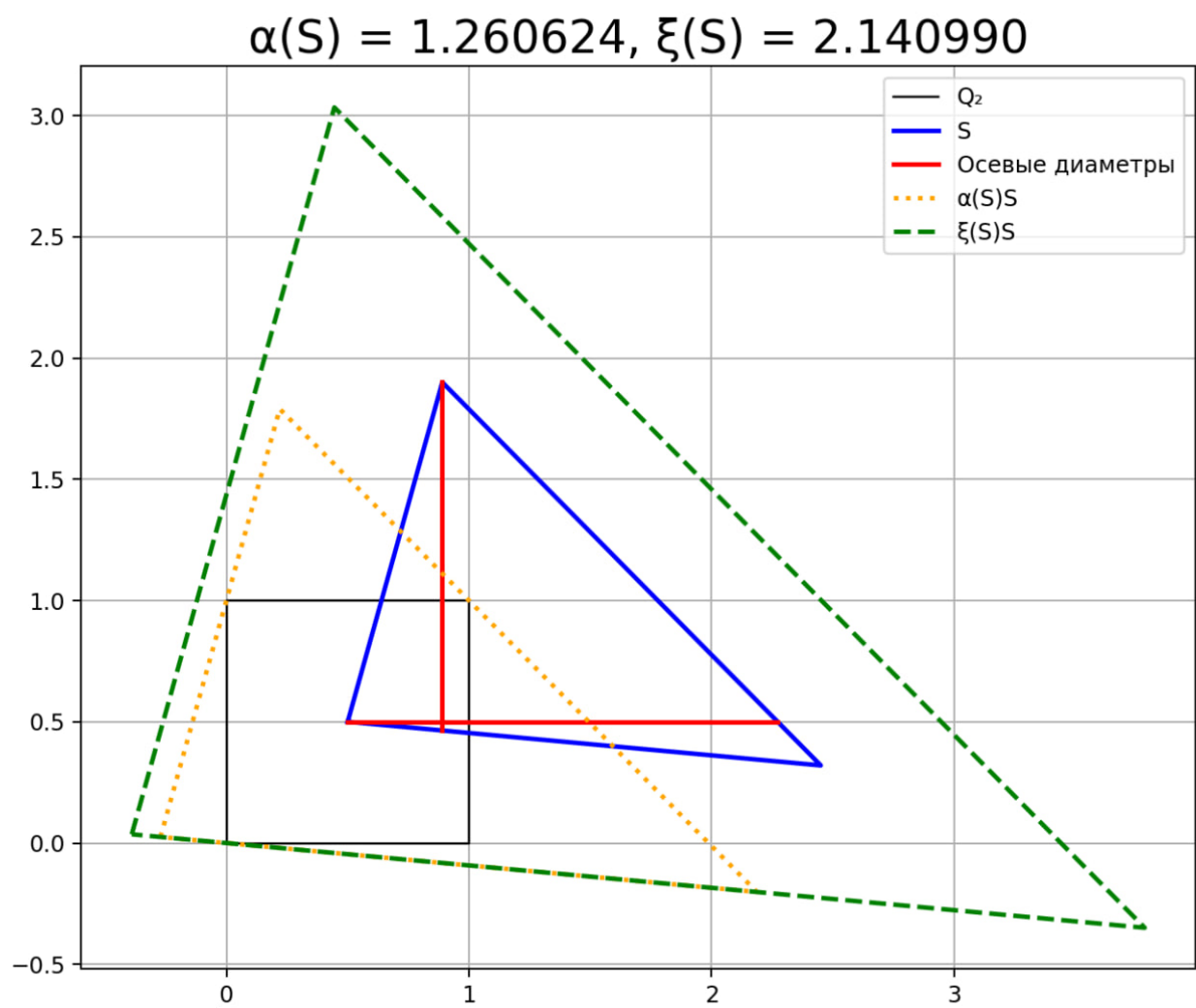


Рис. 7: S частично в Q_2 ($[0, 1]^2$)

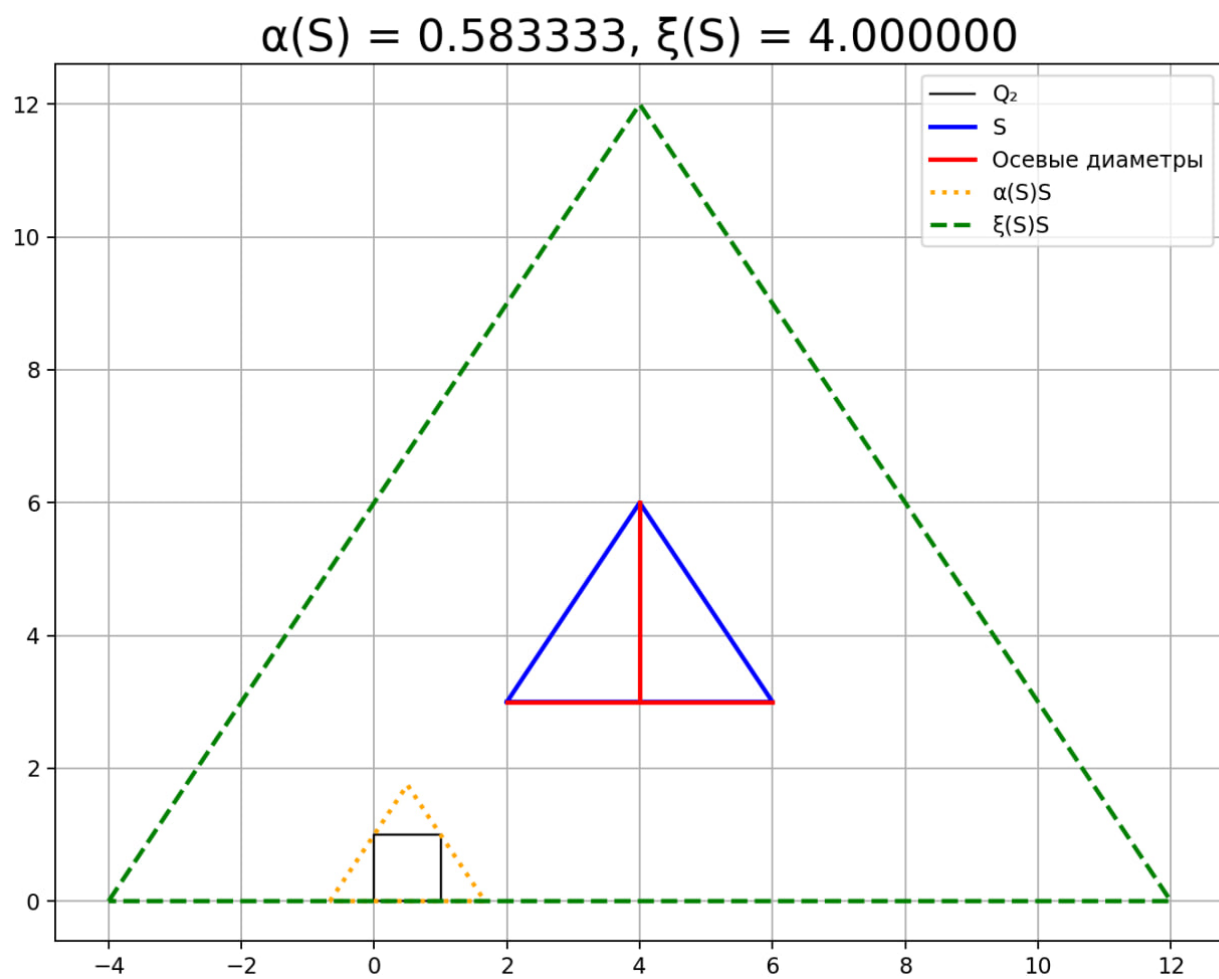


Рис. 8: S вне Q_2 ($[0, 1]^2$)

$$\alpha(S) = 0.272727, \xi(S) = 815.454545$$

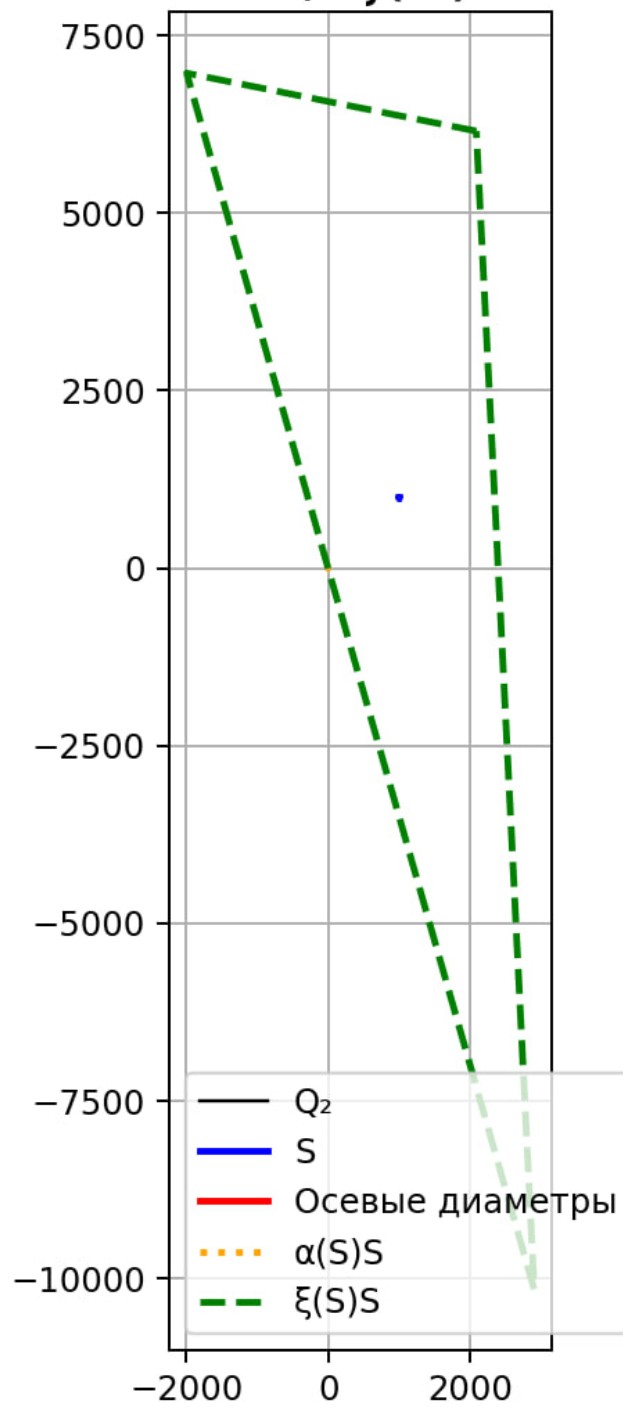


Рис. 9: S далеко вне Q_2 ($[0, 1]^2$) (1)

$$\alpha(S) = 0.272727, \xi(S) = 815.454545$$

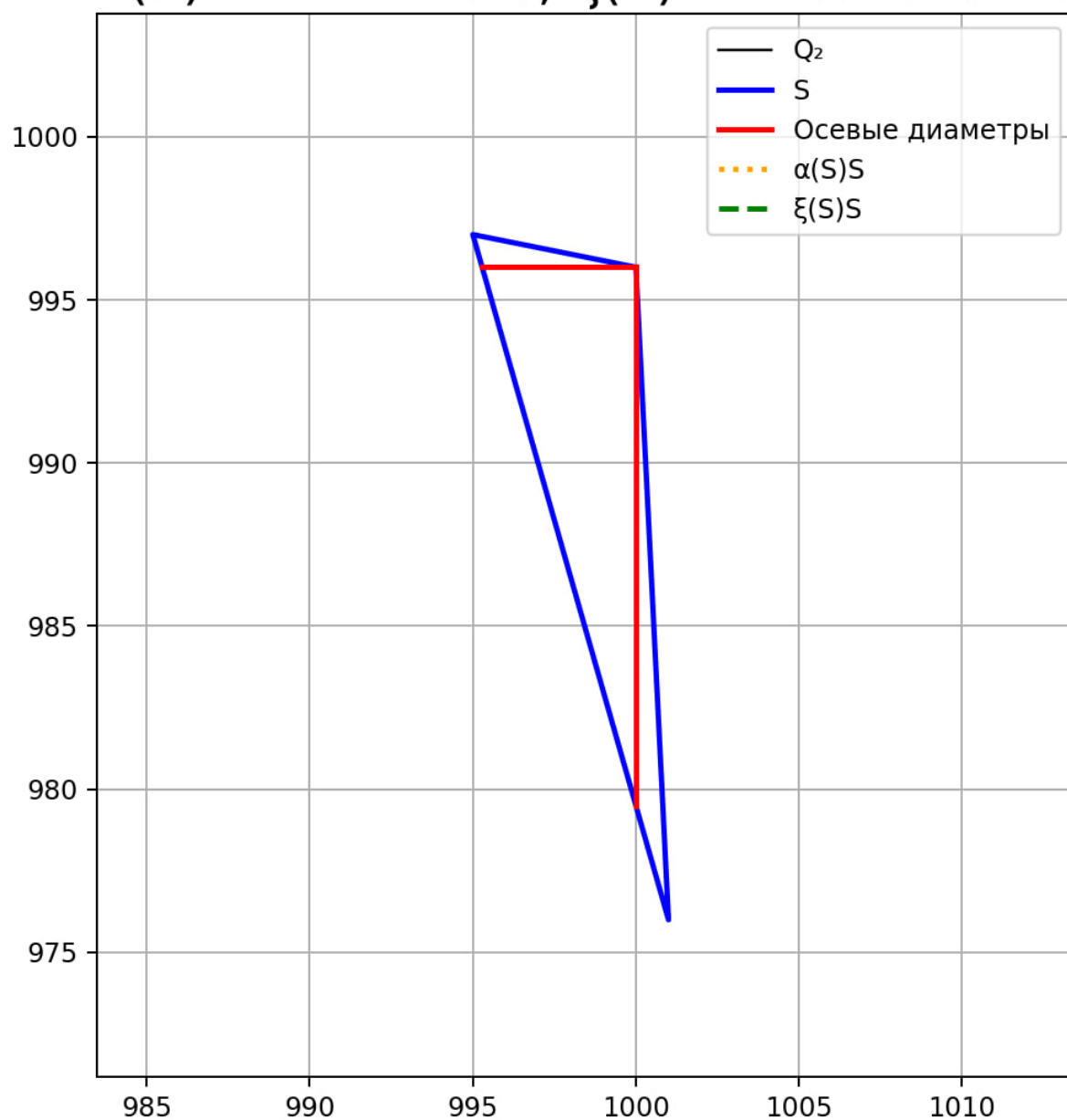


Рис. 10: S далеко вне Q_2 ($[0, 1]^2$) (2)

$$\alpha(S) = 0.272727, \xi(S) = 815.454545$$

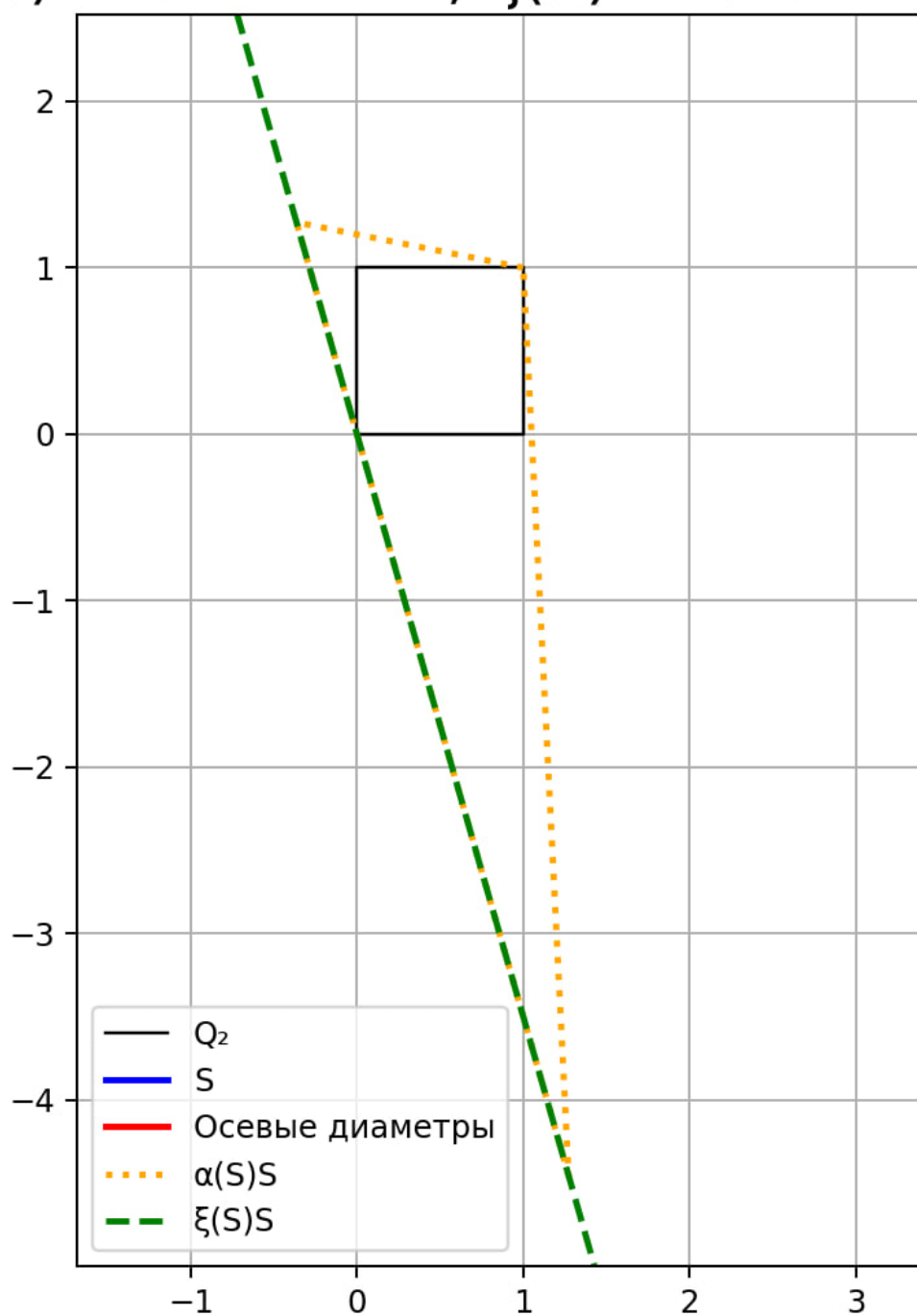


Рис. 11: S далеко вне Q_2 ($[0, 1]^2$) (3)

5.3 Примеры для $\mathbb{R}^3$

$$\alpha(S) = 3.000000, \xi(S) = 3.000000$$

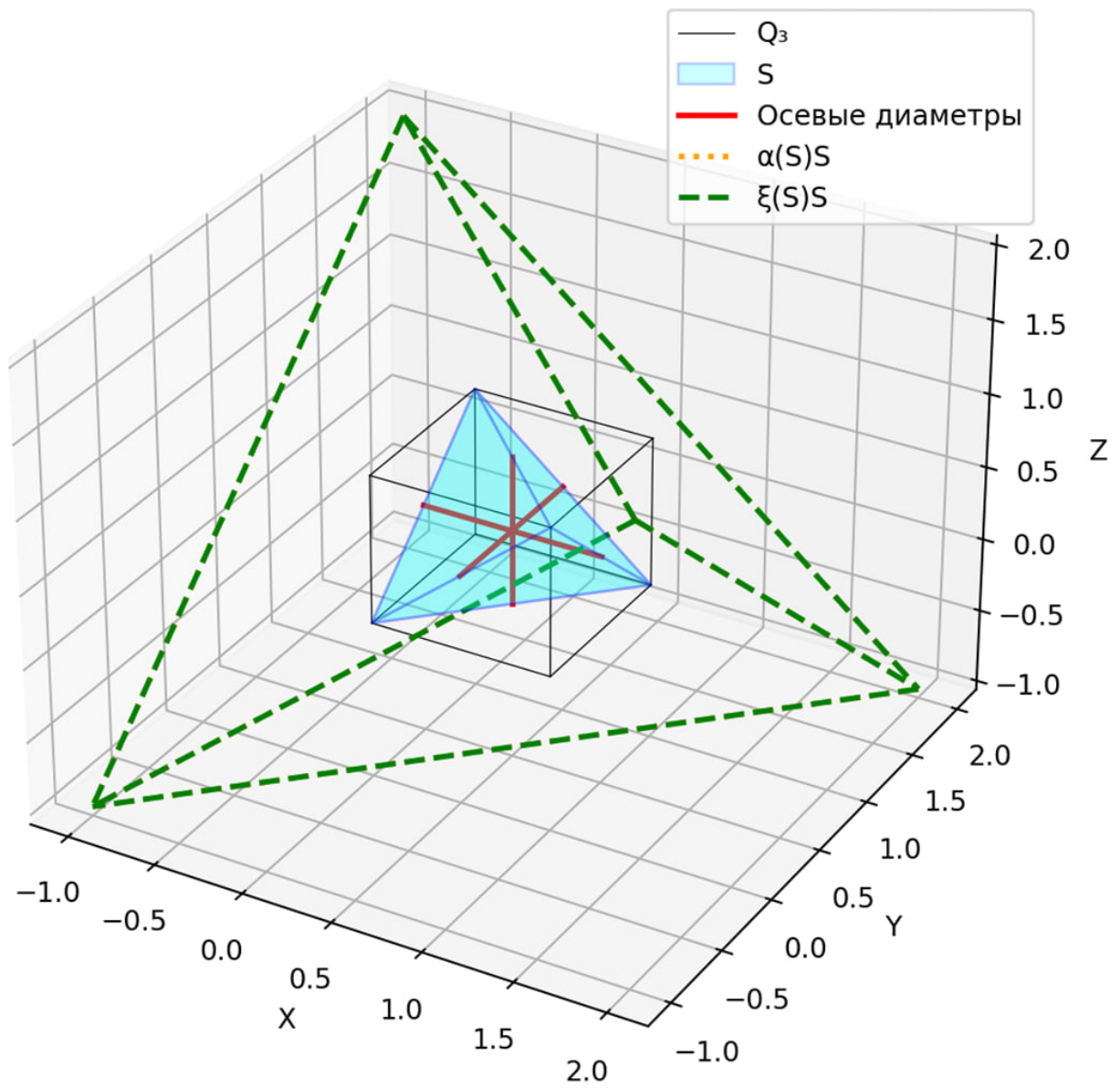


Рис. 12: S внутри Q_3 ($[0, 1]^3$)

$$\alpha(S) = 10.307320, \xi(S) = 14.219411$$

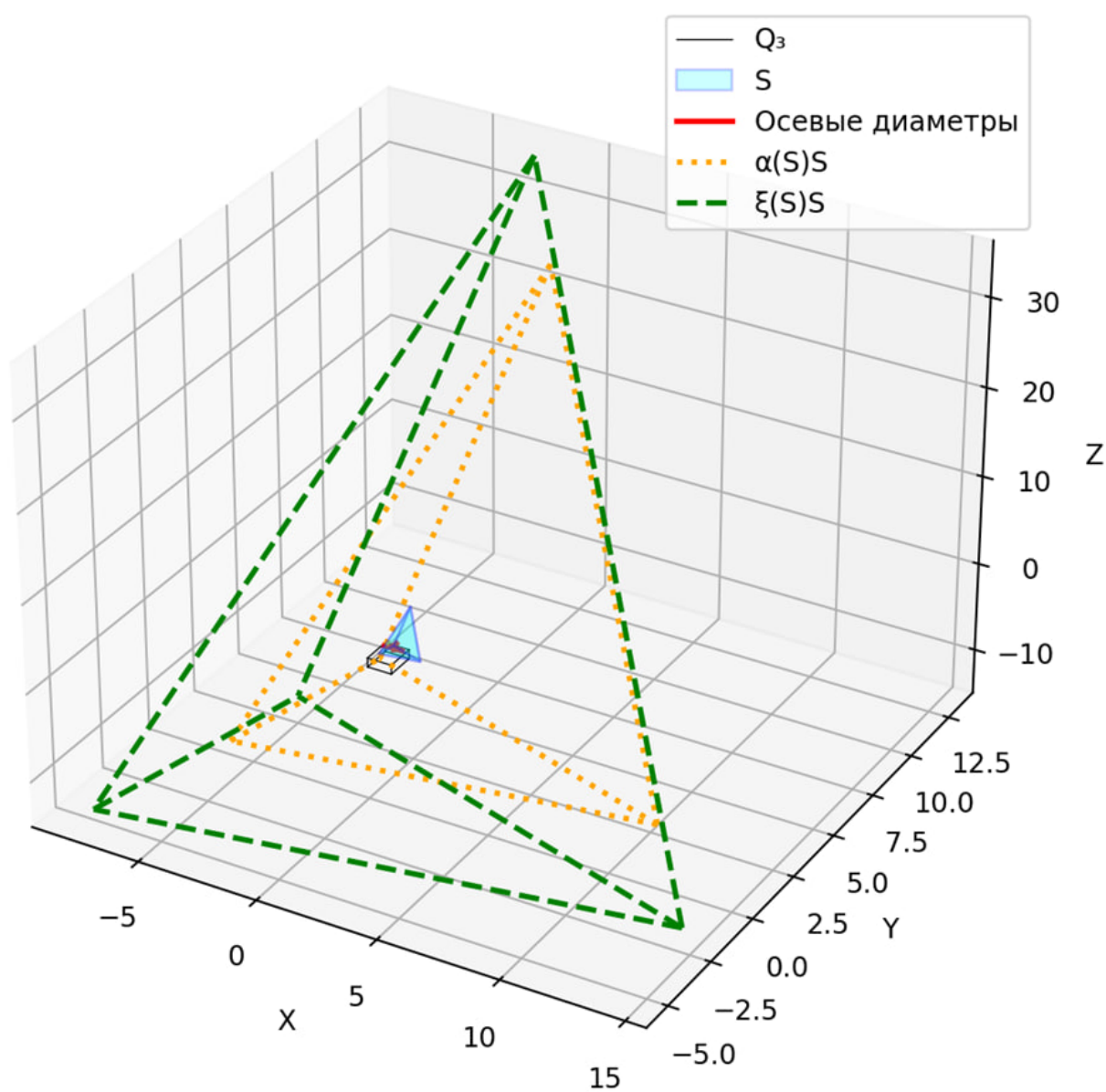


Рис. 13: S частично покрывает Q_3 ($[0, 1]^3$) (1)

$$\alpha(S) = 10.307320, \xi(S) = 14.219411$$

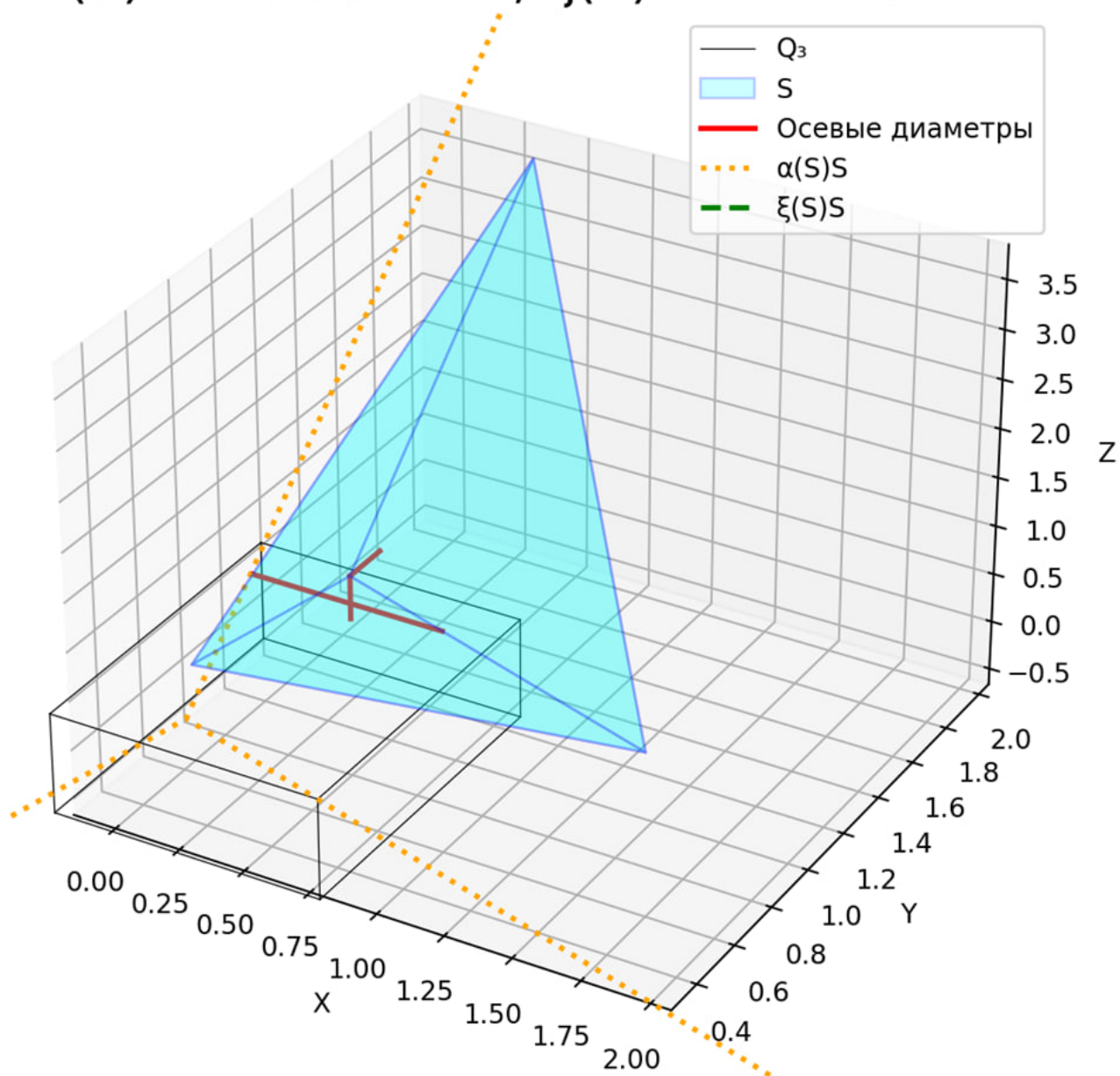


Рис. 14: S частично покрывает Q_3 ($[0, 1]^3$) (2)

$$\alpha(S) = 2.505396, \xi(S) = 10.288856$$

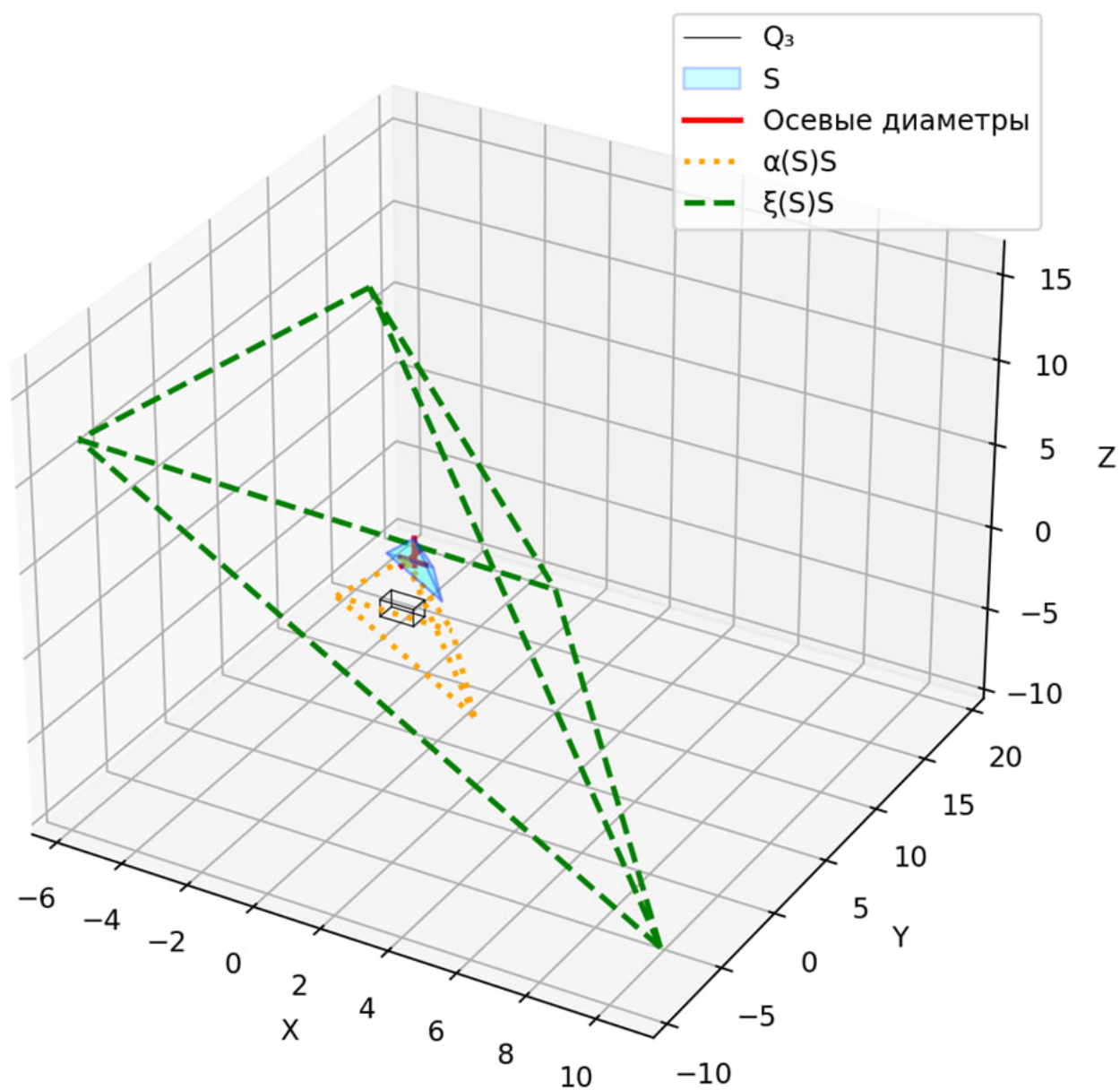


Рис. 15: S вне Q_3 ($[0, 1]^3$) (1)

$$\alpha(S) = 2.505396, \xi(S) = 10.288856$$

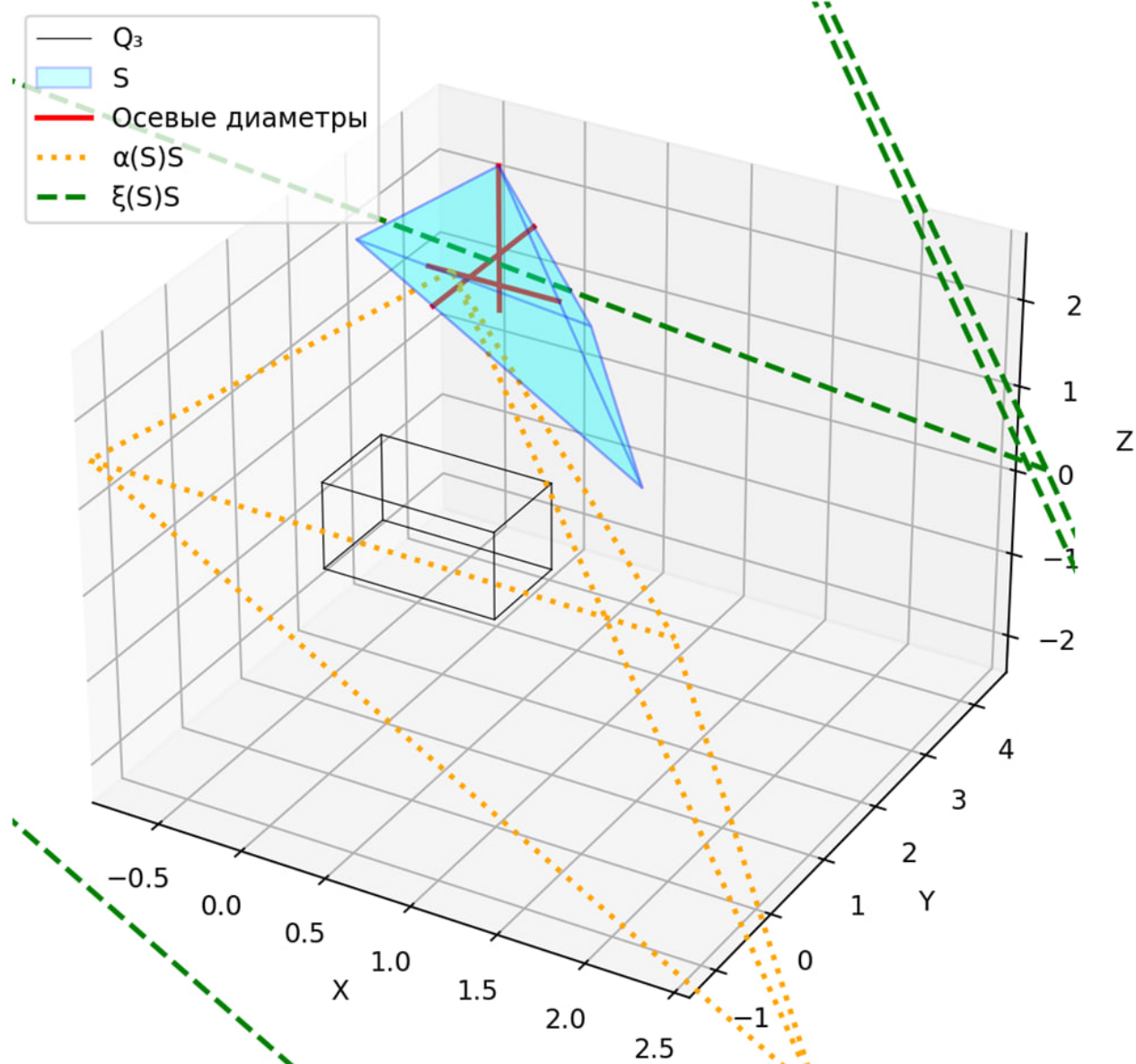


Рис. 16: S вне Q_3 ($[0, 1]^3$) (2)

$$\alpha(S) = 0.233884, \xi(S) = 1.033058$$

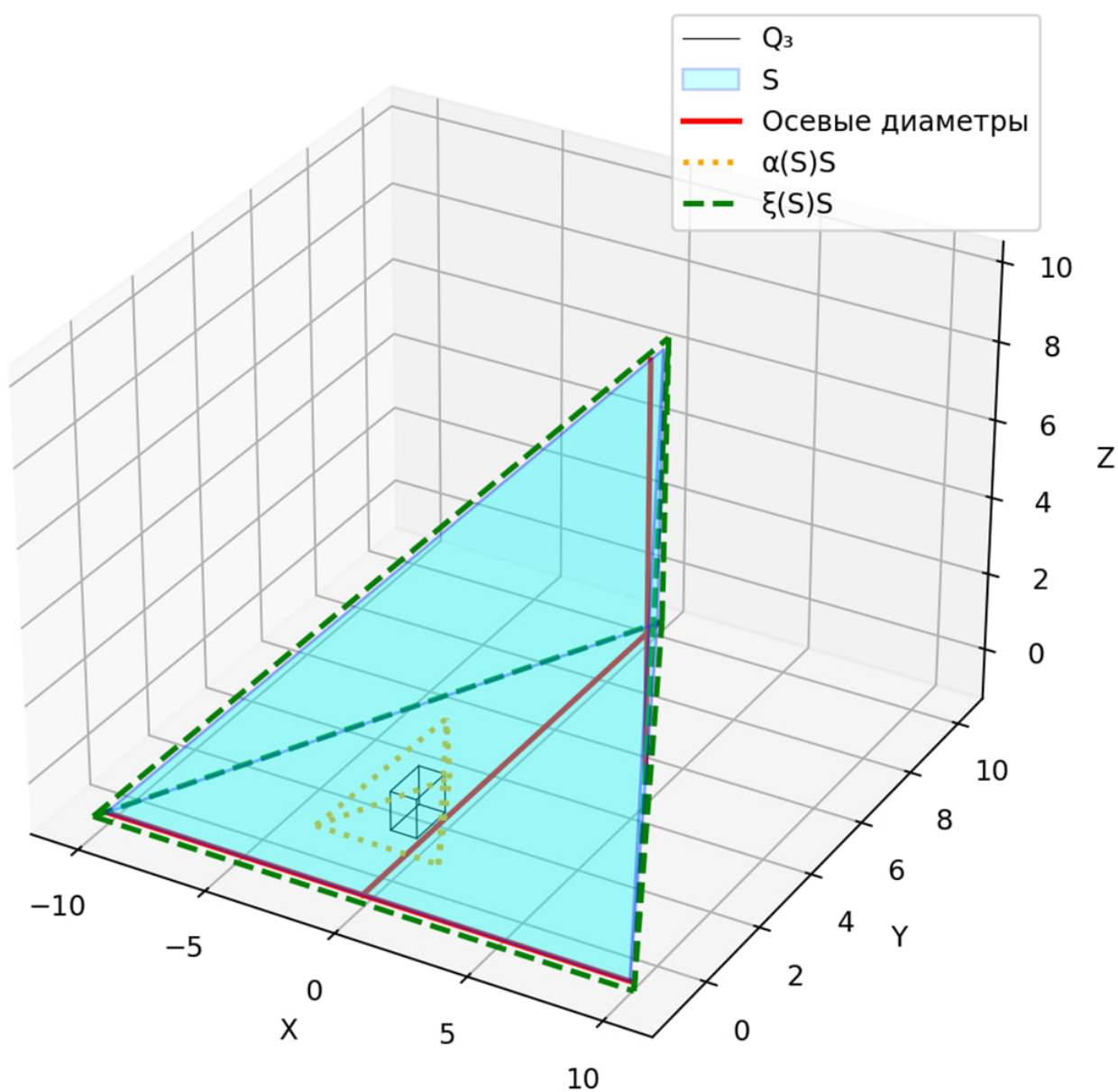


Рис. 17: S покрывает Q_3 ($[0, 1]^3$)

5.4 Примеры для $\mathbb{R}^5$

```
PS D:\PycharmProjects\Laba> python main.py examples/5d.txt
*****
Размерность n = 5
Вершины:
x1 = (0.5, 1, 0.333333, 1, 1)
x2 = (0.5, 0, 0.333333, 1, 1)
x3 = (0.5, 0.5, 0.333333, 0, 1)
x4 = (0.5, 0.5, 0, 0.333333, 0)
x5 = (0, 0.5, 1, 0.333333, 0)
x6 = (1, 0.5, 1, 0.333333, 0)
*****
Строим матрицу A
A:
0.500    1.000    0.333    1.000    1.000    1.000
0.500    0.000    0.333    1.000    1.000    1.000
0.500    0.500    0.333    0.000    1.000    1.000
0.500    0.500    0.000    0.333    0.000    1.000
0.000    0.500    1.000    0.333    0.000    1.000
1.000    0.500    1.000    0.333    0.000    1.000
*****
Ищем обратную матрицу для A
A-1:
0.000    0.000   -0.000    0.000   -1.000    1.000
1.000   -1.000   -0.000   -0.000   -0.000   -0.000
-0.000    0.000    0.000   -1.000    0.500    0.500
0.500    0.500   -1.000    0.000    0.000    0.000
0.167    0.167    0.667   -0.667   -0.167   -0.167
-0.667    0.333    0.333    1.000    0.500   -0.500
*****
```

Рис. 18: S внутри Q_5 ($[0, 1]^5$) (1)

```
*****
Вычисляем  $\alpha(S)$  и  $\xi(S)$ 
 $\alpha(S) = 5.0$ 
 $\xi(S) = 5.000002$ 
*****
Находим осевые диаметры и центр тяжести
Центр тяжести: [0.5    0.5    0.49999983 0.49999983 0.5    ]
*****
Программа отработала за 0.002582 с
*****
❌ Не могу визуализировать для размерности n > 3.
PS D:\PycharmProjects\Laba> |
```

Рис. 19: S внутри Q_5 ($[0, 1]^5$) (2)

6 Заключение

В ходе лабораторной работы создано программное обеспечение, полностью реализующее вычисление ключевых числовых характеристик симплекса – коэффициентов поглощения $\alpha(S)$ и $\xi(S)$ – на основе формул и теорем из учебного пособия М.В. Невского и А.Ю. Ухалова. Программа поддерживает произвольную размерность, а для $n \leq 3$ предоставляет наглядную визуализацию, позволяющую увидеть геометрический смысл этих величин.

Результаты тестирования на примерах из пособия полностью соответствуют ожидаемым. Программа может быть использована как в учебных целях (для изучения свойств симплексов), так и для исследовательских задач, связанных с поиском экстремальных симплексов и проверкой гипотез о значениях ξ_n .

Исходный код на Python оформлен в виде модулей, содержит подробные комментарии и доступен для дальнейшего расширения (например, поддержка других выпуклых тел или более сложных визуализаций).